

**BỘ GIÁO DỤC VÀ ĐÀO TẠO
TRƯỜNG ĐẠI HỌC VĂN HIẾN**



**BÁO CÁO CUỐI KỲ
HỌC PHẦN BẢO MẬT TRONG IOT**

ĐỀ TÀI: VAI TRÒ CỦA MẬT MÃ TRONG BẢO MẬT IOT

Lớp học phần	253INT441001
Giảng viên	Hồ Nhật Minh
Sinh viên	Nguyễn Phước Hậu
MSSV	231A010774
Mã đề tài	36
Hướng đề tài	F
Link repo GitHub	https://github.com/phuchochau14/231A010774-NguyenPhuocHau-DeTai36-BaoMatIoT
Ngày nộp	03/08/2026

TP.HCM, THÁNG 8 NĂM 2026

**BỘ GIÁO DỤC VÀ ĐÀO TẠO
TRƯỜNG ĐẠI HỌC VĂN HIẾN**

-----❧*❧-----



**BÁO CÁO CUỐI KỲ
HỌC PHẦN BẢO MẬT TRONG IOT**

ĐỀ TÀI: VAI TRÒ CỦA MẬT MÃ TRONG BẢO MẬT IOT

Lớp học phần	253INT441001
Giảng viên	Hồ Nhật Minh
Sinh viên	Nguyễn Phước Hậu
MSSV	231A010774
Mã đề tài	36
Hường đề tài	F
Link repo GitHub	https://github.com/phuochau14/231A010774-NguyenPhuocHau-DeTai36-BaoMatIoT
Ngày nộp	03/08/2026

TP.HCM, THÁNG 8 NĂM 2026

LỜI CẢM ƠN

Trong quá trình thực hiện tiểu luận “Vai trò của mật mã trong bảo mật IoT”, em đã nhận được rất nhiều sự hướng dẫn, hỗ trợ và động viên quý báu để có thể hoàn thành bài làm một cách trọn vẹn nhất.

Trước hết, em xin gửi lời cảm ơn chân thành và sâu sắc nhất đến thầy **Hồ Nhựt Minh**, giảng viên phụ trách học phần Bảo mật IoT (INT4410), đã tận tình giảng dạy, theo sát tiến độ và góp ý chi tiết qua từng phiếu hướng dẫn trong suốt quá trình em thực hiện đề tài. Những nhận xét cụ thể và định hướng rõ ràng của thầy đã giúp em kịp thời điều chỉnh và hoàn thiện bài làm đúng hạn.

Em cũng xin gửi lời cảm ơn đến Khoa Công nghệ Thông tin và Trường Đại học Văn Hiến đã tạo điều kiện về chương trình học, tài liệu tham khảo và môi trường học tập để em có thể tiếp cận kiến thức thực tiễn về an toàn thông tin trong lĩnh vực IoT.

Cuối cùng, em xin cảm ơn gia đình, bạn bè đã luôn động viên, đồng hành và hỗ trợ em trong suốt thời gian học tập và thực hiện tiểu luận này.

Do thời gian thực hiện có hạn và kiến thức bản thân còn nhiều hạn chế, bài tiểu luận khó tránh khỏi những thiếu sót. Em rất mong nhận được sự góp ý của thầy để bài làm được hoàn thiện hơn.

LỜI CAM ĐOAN

Tôi xin cam đoan rằng đề tài "Vai trò của mật mã trong bảo mật IoT" là kết quả của quá trình tìm hiểu, nghiên cứu và thực hiện của bản thân dưới sự hướng dẫn của giảng viên. Các nội dung trong báo cáo được xây dựng dựa trên kiến thức đã học, các tài liệu chuẩn tham khảo và quá trình triển khai, kiểm thử thực tế trong phạm vi đề tài.

Trong quá trình thực hiện báo cáo, tôi có sử dụng công cụ trí tuệ nhân tạo (AI) để hỗ trợ tìm hiểu kiến thức, gợi ý cách trình bày, diễn đạt nội dung, xây dựng mã nguồn minh họa và rà soát lỗi. Tuy nhiên, toàn bộ nội dung sau khi được AI hỗ trợ đều đã được tôi xem xét, chỉnh sửa, tự chạy thử chương trình để kiểm chứng kết quả và lựa chọn trước khi đưa vào báo cáo. Những phần sử dụng tài liệu từ các nguồn khác đều được trích dẫn theo quy định.

Tôi xin chịu hoàn toàn trách nhiệm về tính trung thực, chính xác của nội dung cũng như các số liệu, kết quả được trình bày trong báo cáo. Trường hợp có sai sót, bao gồm cả những thông tin hoặc số liệu do AI cung cấp nhưng chưa được kiểm chứng đầy đủ, tôi xin hoàn toàn chịu trách nhiệm và chấp nhận mọi hình thức xử lý theo quy định của nhà trường.

TP. Hồ Chí Minh, ngày 03 tháng 08 năm 2026

Sinh viên thực hiện
(Ký tên và ghi rõ họ tên)

[illegible]

Bảng số:

GIẢNG VIÊN CHẤM

V

MỤC LỤC

LỜI CẢM ƠN.....	iii
LỜI CAM ĐOAN.....	iv
CHƯƠNG 1. TỔNG QUAN ĐỀ TÀI	viii
1.1. Bối cảnh và lý do chọn đề tài.....	viii
1.2. Phát biểu vấn đề bảo mật	viii
1.3. Mục tiêu nghiên cứu	ix
1.4. Đối tượng, phạm vi và giới hạn an toàn	ix
1.5. Bảng sản phẩm dự kiến	x
1.6. Cấu trúc báo cáo	x
CHƯƠNG 2. CƠ SỞ LÝ THUYẾT.....	xi
2.1. Kiến trúc/bối cảnh của đề tài	xi
2.2. Các khái niệm bảo mật cơ bản sử dụng trong bài.....	xi
2.3. Trọng tâm: Vai trò của mật mã trong bảo mật IoT (hướng F)	xii
2.4. Chuẩn, công cụ, nguồn GitHub tham chiếu	xiv
2.5. Công trình liên quan.....	xiv
2.6. Tiểu kết.....	xv
CHƯƠNG 3. PHƯƠNG PHÁP VÀ THIẾT KẾ	xvi
3.1. Phương pháp thực hiện	xvi
3.2. Mô hình / kiến trúc đề xuất.....	xvi
3.3. Môi trường, công cụ, dữ liệu.....	xvii
3.4. Quy trình triển khai / đánh giá	xviii
3.5. Tiêu chí đánh giá	xix
CHƯƠNG 4. TRIỂN KHAI VÀ KẾT QUẢ	xx
4.1. Chuẩn bị môi trường	xx
4.2. Thành phần sản phẩm	xx
4.3. Kịch bản thử nghiệm	xxi
4.4. Kết quả.....	xxii
4.5. Script sinh báo cáo (chương trình minh họa).....	xxiv
4.6. Thảo luận và hạn chế.....	xxiv
CHƯƠNG 5. ĐÁNH GIÁ BẢO MẬT	xxvi

5.1. Tài sản và dữ liệu	xxvi
5.2. Mối đe dọa và lỗ hổng (ánh xạ STRIDE)	xxvi
5.3. Ma trận rủi ro.....	xxvii
5.4. Biện pháp giảm thiểu	xxviii
5.5. Rủi ro còn lại	xxviii
CHƯƠNG 6. KẾT LUẬN.....	xxx
6.1. Kết luận theo mục tiêu.....	xxx
6.2. Hạn chế	xxx
6.3. Hướng phát triển.....	xxx
DANH MỤC TÀI LIỆU THAM KHẢO	xxxi
PHỤ LỤC - PHẦN A: CHECKLIST TIẾN ĐỘ THEO TUẦN	xxxii

Danh mục bảng

Bảng 1.1.Mục nghiên cứu.....	ix
Bảng 1.5. Sản phẩm dự kiến	x
Bảng 2.1 tổng hợp và so sánh 5 thuật toán/kỹ thuật được dùng trong đề tài	xiii
Bảng 2.4. Chuẩn, công cụ, nguồn Github tham chiếu	xiv
Bảng 3.3. Môi trường, công cụ, dữ liệu.....	xvii
Bảng 3.4. Quy trình triển khai/đánh giá	xviii
Bảng 4.3. Kịch bản thử nghiệm	xxi
Bảng 4.4. Kết quả	xxii
Bảng 5.1. Tài sản/giá trị/yêu cầu	xxvi
Bảng 5.3. Ma trận rủi ro	xxvii
Bảng checklist tiến độ theo tuần.....	xxxii

Danh mục hình ảnh

Hình 2.1 - Kiến trúc hệ thống và ranh giới tin cậy của đề tài.....	xi
Hình 2.2 - Vòng đời khóa mật mã (key lifecycle) trong IoT	xiii
Hình 3.1 - Mô hình đề xuất: áp dụng kỹ thuật mật mã theo từng giai đoạn	xvii

CHƯƠNG 1. TỔNG QUAN ĐỀ TÀI

1.1. Bối cảnh và lý do chọn đề tài

Các hệ thống IoT (Internet of Things) hiện diện ngày càng rộng trong công nghiệp, nông nghiệp thông minh, nhà thông minh và giám sát môi trường. Đặc điểm chung của các thiết bị này là tài nguyên hạn chế (CPU, RAM, pin) nhưng lại thường xuyên truyền dữ liệu nhạy cảm qua các kênh không đáng tin cậy (mạng không dây, gateway trung gian, broker công cộng). Nếu không có cơ chế mật mã phù hợp, dữ liệu cảm biến có thể bị nghe lén, sửa đổi, hoặc giả mạo nguồn gốc mà hệ thống không hề hay biết.

Đề tài 36 được chọn vì mật mã học chính là lớp phòng thủ nền tảng nhất trong bảo mật IoT: dù thiết kế mạng hay chính sách truy cập có chặt chẽ đến đâu, nếu thiếu cơ chế bảo vệ tính bí mật, toàn vẹn và xác thực ở tầng dữ liệu, hệ thống vẫn có thể bị khai thác. Việc hiểu đúng vai trò và giới hạn của từng kỹ thuật mật mã (hash, HMAC, mã hóa đối xứng, chữ ký số) giúp người thiết kế hệ thống chọn đúng công cụ cho đúng bài toán, tránh tình trạng dùng sai hoặc dùng thừa gây lãng phí tài nguyên.

1.2. Phát biểu vấn đề bảo mật

Kịch bản minh họa: một nút cảm biến (sensor node) đo nhiệt độ/độ ẩm trong kho hàng, gửi dữ liệu qua gateway MQTT lên cloud server để cảnh báo khi vượt ngưỡng an toàn. Nếu kênh truyền không được bảo vệ bằng mật mã, một kẻ tấn công đứng giữa (Man-in-the-Middle) có thể:

- Đọc trộm giá trị cảm biến (mất tính bí mật - Confidentiality);
- Sửa đổi giá trị nhiệt độ trước khi đến server, ví dụ hạ giá trị thật để che giấu sự cố quá nhiệt (mất tính toàn vẹn - Integrity);
- Giả danh một sensor node hợp lệ để gửi dữ liệu rác hoặc lệnh điều khiển giả (mất tính xác thực - Authenticity).

Vấn đề đặt ra: cần xác định lớp mật mã nào giải quyết được từng nguy cơ trên, minh họa bằng chương trình thực nghiệm, và đánh giá rủi ro khi triển khai sai.

1.3. Mục tiêu nghiên cứu

Bảng 1.1.Mục nghiên cứu

Mã mục tiêu	Nội dung mục tiêu	Minh chứng đạt được
MT-01	Giải thích đúng vai trò của 4 kỹ thuật mật mã (hash, HMAC, mã hóa đối xứng, chữ ký số) trong bảo vệ dữ liệu IoT.	Chương 2
MT-02	Xây dựng mô hình áp dụng mật mã vào luồng dữ liệu cảm biến cụ thể của đề tài.	Chương 3 (Hình 3.1)
MT-03	Cài đặt chương trình Python minh họa hash/HMAC/AES-GCM/chữ ký số, phát hiện giả mạo qua tối thiểu 5 kịch bản thử nghiệm.	Chương 4
MT-04	Phân tích rủi ro bảo mật (tài sản, đe dọa STRIDE, ma trận rủi ro) và đề xuất biện pháp giảm thiểu tương ứng.	Chương 5

1.4. Đối tượng, phạm vi và giới hạn an toàn

Đối tượng nghiên cứu: các kỹ thuật mật mã ứng dụng cho dữ liệu cảm biến IoT (hash SHA-256, HMAC-SHA256, AES-256-GCM, chữ ký số RSA-PSS/ECDSA).

Phạm vi: mô phỏng phần mềm trên máy tính thông thường bằng Python, dữ liệu sensor ở dạng JSON giả lập; đối chiếu lý thuyết với các thư viện/công cụ mã nguồn mở tiêu chuẩn công nghiệp (Mbed TLS, TinyCrypt) và bộ chuẩn OWASP ISVS.

Giới hạn an toàn: đề tài chỉ thực hiện trong môi trường lab/mô phỏng, không tấn công hoặc rà quét bất kỳ hệ thống thật nào không thuộc sở hữu hoặc không có sự cho phép của người thực hiện.

1.5. Bảng sản phẩm dự kiến

Bảng 1.5. Sản phẩm dự kiến

Mục tiêu	Sản phẩm đầu ra	Cách kiểm chứng	Chương trình bày
MT-01	Nội dung lý thuyết + bảng so sánh thuật toán	Đối chiếu tài liệu chuẩn (NIST FIPS 198-1, SP 800-38D)	Chương 2
MT-02	Sơ đồ kiến trúc + mô hình đề xuất	Phản biện với GV, đối chiếu use-case thực tế	Chương 3
MT-03	Code Python + log chạy thực tế	Chạy lại script, đối chiếu log trước/sau tấn công	Chương 4
MT-04	Bảng tài sản, STRIDE, ma trận rủi ro, biện pháp	Đối chiếu OWASP ISVS V4/V8	Chương 5

1.6. Cấu trúc báo cáo

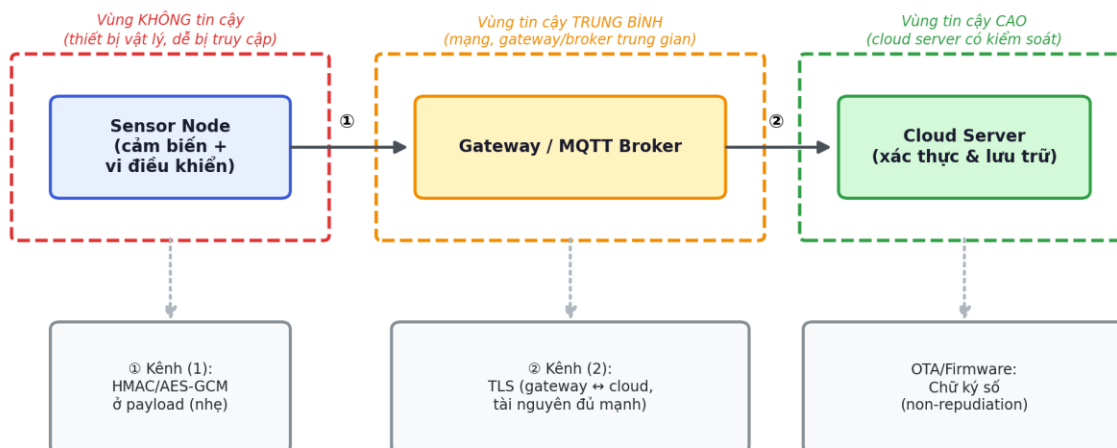
Báo cáo gồm 6 chương: Chương 1 giới thiệu bối cảnh và mục tiêu; Chương 2 trình bày cơ sở lý thuyết về mật mã trong IoT; Chương 3 mô tả phương pháp và mô hình đề xuất; Chương 4 trình bày quá trình triển khai và kết quả thử nghiệm; Chương 5 đánh giá rủi ro bảo mật; Chương 6 tổng kết và đề xuất hướng phát triển.

CHƯƠNG 2. CƠ SỞ LÝ THUYẾT

2.1. Kiến trúc/bối cảnh của đề tài

Hệ thống minh họa trong đề tài gồm 3 vùng tin cậy khác nhau, phản ánh đúng kiến trúc IoT phổ biến: (1) Sensor Node - thiết bị vật lý, dễ bị truy cập trái phép nên được xem là vùng KHÔNG tin cậy; (2) Gateway/MQTT Broker - điểm trung chuyển, tin cậy ở mức trung bình vì có thể do bên thứ ba vận hành; (3) Cloud Server - vùng tin cậy cao do có kiểm soát truy cập chặt chẽ. Mỗi ranh giới tin cậy (trust boundary) là nơi cần áp dụng cơ chế mật mã tương ứng.

Hình 2.1 — Kiến trúc hệ thống và ranh giới tin cậy (trust boundary)



Hình 2.1 - Kiến trúc hệ thống và ranh giới tin cậy của đề tài

2.2. Các khái niệm bảo mật cơ bản sử dụng trong bài

- Tài sản (asset): dữ liệu cảm biến, khóa mật mã, firmware, danh tính thiết bị - những gì cần được bảo vệ.
- Mối đe dọa (threat) và lỗ hổng (vulnerability): mối đe dọa là hành vi có khả năng gây hại (ví dụ giả mạo gói tin); lỗ hổng là điểm yếu bị lợi dụng (ví dụ thiếu HMAC khiến dữ liệu có thể bị sửa mà không bị phát hiện).
- Rủi ro (risk): xác suất mối đe dọa khai thác thành công lỗ hổng, nhân với mức độ ảnh hưởng - được lượng hóa trong ma trận rủi ro ở Chương 5.

- Bộ ba CIA: Confidentiality (bí mật) - dữ liệu chỉ bên được phép mới đọc được; Integrity (toàn vẹn) - dữ liệu không bị sửa đổi trái phép; Availability (sẵn sàng) - hệ thống vẫn hoạt động khi bị tấn công từ chối dịch vụ.
- Xác thực (authentication) và không thể chối bỏ (non-repudiation): xác thực chứng minh đúng bên gửi; non-repudiation khiến bên gửi không thể phủ nhận đã tạo ra dữ liệu/firmware đó - chỉ chữ ký số bất đối xứng mới cung cấp được thuộc tính này.
- STRIDE: mô hình phân loại đe dọa (Spoofing, Tampering, Repudiation, Information Disclosure, Denial of Service, Elevation of Privilege), được dùng để ánh xạ đe dọa cụ thể ở Chương 5.

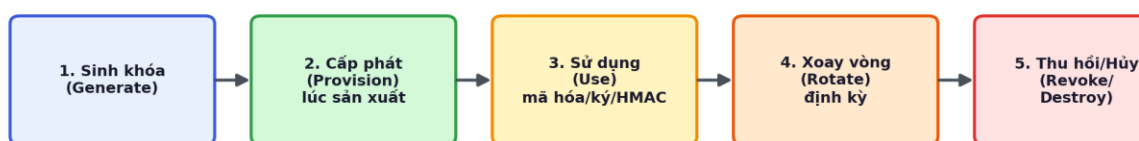
2.3. Trọng tâm: Vai trò của mật mã trong bảo mật IoT (hướng F)

Bốn kỹ thuật mật mã cốt lõi được sử dụng trong đề tài, mỗi kỹ thuật giải quyết một hoặc nhiều mục tiêu bảo mật khác nhau:

- Hash (SHA-256): sinh ra một chuỗi đại diện cố định độ dài từ dữ liệu đầu vào bất kỳ. Không cần khóa, ai cũng tính được - chỉ đảm bảo phát hiện thay đổi (toàn vẹn), KHÔNG đảm bảo được ai là người tạo ra dữ liệu (không có xác thực).
- HMAC (HMAC-SHA256): kết hợp hash với một khóa bí mật dùng chung giữa hai bên. Vì cần khóa để tạo mã hợp lệ, HMAC đảm bảo cả toàn vẹn LẦN xác thực nguồn gốc - phù hợp cho thiết bị tài nguyên thấp vì tính toán nhẹ.
- Mã hóa đối xứng có xác thực (AES-256-GCM): che giấu nội dung dữ liệu (bí mật) đồng thời tạo ra thẻ xác thực (authentication tag) tích hợp sẵn để phát hiện giả mạo (toàn vẹn) - không cần ghép thêm HMAC riêng.
- Chữ ký số (RSA-PSS / ECDSA): dùng cặp khóa bất đối xứng - bên ký (thường là nhà sản xuất) giữ private key, các thiết bị chỉ giữ public key để xác minh. Ngoài toàn vẹn và xác thực, chữ ký số còn cung cấp tính không thể chối bỏ, phù hợp cho các sự kiện quan trọng như ký firmware/OTA update, nhưng KHÔNG phù hợp để ký từng gói tin cảm biến vì tốn tài nguyên hơn HMAC rất nhiều lần.

Việc lựa chọn và sử dụng đúng khóa xuyên suốt vòng đời (sinh khóa → cấp phát → sử dụng → xoay vòng → thu hồi) quyết định phần lớn mức độ an toàn thực tế của hệ thống, bất kể thuật toán mạnh đến đâu:

Hình 2.2 — Vòng đời khóa mật mã (key lifecycle) trong IoT



Mỗi thiết bị có khóa/định danh RIÊNG — key bị lộ ở bước nào cũng có thể thu hồi (bước 5) mà không ảnh hưởng các thiết bị khác trong hệ thống.

Hình 2.2 - Vòng đời khóa mật mã (key lifecycle) trong IoT

Bảng 2.1 tổng hợp và so sánh 5 thuật toán/kỹ thuật được dùng trong đề tài

Thuật toán/Kỹ thuật	Loại khóa	Mục tiêu bảo mật	Kích thước khóa	Tốc độ / tài nguyên	Use case IoT điển hình
SHA-256	Không cần khóa	Toàn vẹn		Rất nhanh, rất nhẹ	Kiểm tra checksum cơ bản
HMAC-SHA256	Đối xứng (chung)	Toàn vẹn + Xác thực	256-bit	Nhanh, nhẹ	Xác thực gói tin cảm biến
AES-256-GCM	Đối xứng (chung)	Bí mật + Toàn vẹn	256-bit	Nhanh (có hỗ trợ phần cứng)	Mã hóa payload/kênh truyền
RSA-2048 (PSS)	Bất đối xứng	Xác thực + Non-repudiation	2048-bit	Chậm (bậc ms)	Ký firmware / OTA update
ECDSA (P-256)	Bất đối xứng	Xác thực + Non-repudiation	256-bit	Nhanh hơn RSA cùng mức an toàn	Thiết bị hạn chế tài nguyên hơn

2.4. Chuẩn, công cụ, nguồn GitHub tham chiếu

Bảng 2.4. Chuẩn, công cụ, nguồn Github tham chiếu

Tên	Link	Ngày truy cập	Phần đã sử dụng
Mbed TLS	https://github.com/Mbed-TLS/mbedtls	28/07/2026	Tham chiếu cách triển khai AES-GCM/TLS chuẩn công nghiệp cho thiết bị nhúng (mục 2.3, 3.3)
TinyCrypt	https://github.com/intel/tinycrypt	28/07/2026	Tham chiếu thư viện mật mã nhẹ (SHA-256, HMAC, ECDSA) cho MCU tài nguyên hạn chế (mục 2.3, 3.3)
OWASP ISVS	https://github.com/OWASP/IoT-Security-Verification-Standard-ISVS	28/07/2026	Đối chiếu yêu cầu V4 (Communication) và V8 (Cryptography) trong Chương 5

2.5. Công trình liên quan

(1) Mbed TLS là thư viện TLS/mật mã mã nguồn mở của ARM, được dùng rộng rãi trong thiết bị nhúng thương mại; điểm mạnh là đầy đủ bộ giao thức TLS và các cipher suite chuẩn, nhưng đòi hỏi tài nguyên (RAM/ROM) lớn hơn so với các thư viện tối giản. Đề tài kế thừa cách Mbed TLS tổ chức API mã hóa AEAD (AES-GCM) làm tham chiếu thiết kế.

(2) TinyCrypt (Intel) là thư viện mật mã tối giản, chỉ cài đặt các nguyên thủy cần thiết (hash, HMAC, ECDSA, AES) cho vi điều khiển hạn chế tài nguyên, không có toàn bộ giao thức TLS. Đề tài kế thừa triết lý “tối giản, đúng chuẩn” này khi lựa chọn thuật toán ở Bảng 2.1.

(3) OWASP IoT Security Verification Standard (ISVS) là bộ checklist xác minh bảo mật IoT theo từng nhóm yêu cầu (kiến trúc, giao tiếp, mật mã, quản lý danh tính...). Đề tài kế thừa nhóm yêu cầu V4 và V8 làm khung đối chiếu khi đánh giá rủi ro ở Chương 5, thay vì tự đề xuất tiêu chí đánh giá riêng.

2.6. Tiểu kết

Chương 2 đã trình bày bốn kỹ thuật mật mã cốt lõi (hash, HMAC, mã hóa đối xứng AEAD, chữ ký số) cùng vai trò riêng biệt của từng kỹ thuật đối với ba mục tiêu bảo mật bí mật - toàn vẹn - xác thực, đồng thời nhấn mạnh vai trò quyết định của việc quản lý vòng đời khóa. Đây là nền tảng lý thuyết để Chương 3 xây dựng mô hình đề xuất áp dụng cụ thể các kỹ thuật này vào luồng dữ liệu cảm biến của đề tài.

CHƯƠNG 3. PHƯƠNG PHÁP VÀ THIẾT KẾ

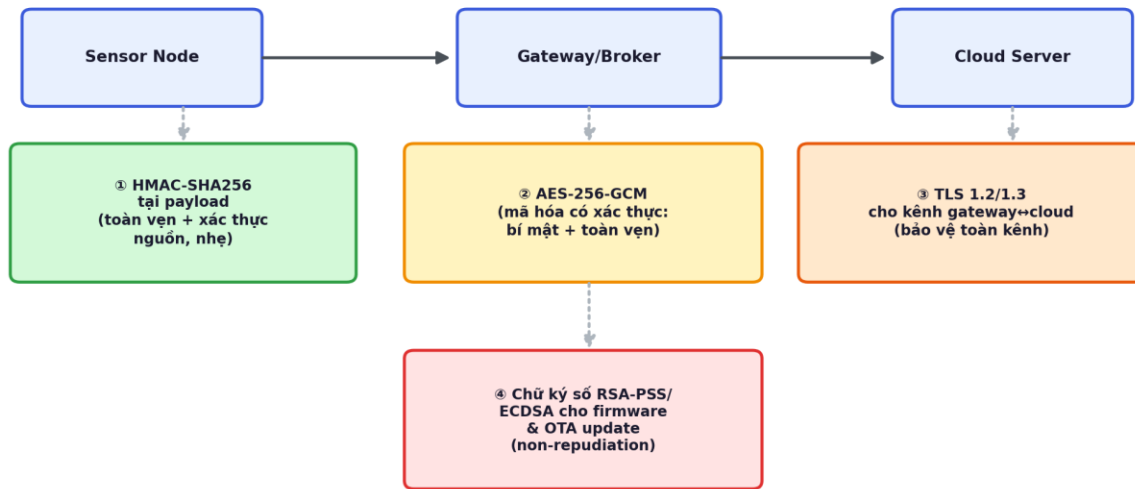
3.1. Phương pháp thực hiện

- Bước 1 - Khảo sát: tổng hợp lý thuyết về 4 kỹ thuật mật mã và đối chiếu với 3 nguồn GitHub bắt buộc (Mbed TLS, TinyCrypt, OWASP ISVS) để xác định thuật toán/thư viện phù hợp cho từng vai trò.
- Bước 2 - Thiết kế mô hình: xây dựng sơ đồ áp dụng mật mã theo từng giai đoạn của luồng dữ liệu (Hình 3.1), xác định rõ vị trí dùng HMAC, AES-GCM, TLS và chữ ký số.
- Bước 3 - Dựng dữ liệu & môi trường mô phỏng: tạo dữ liệu sensor JSON giả lập (device_id, nhiệt độ, độ ẩm, điện áp pin, số thứ tự gói tin) làm đầu vào thống nhất cho toàn bộ thử nghiệm.
- Bước 4 - Cài đặt chương trình: viết script Python (crypto_demo.py) cài đặt SHA-256, HMAC-SHA256, AES-256-GCM và chữ ký số RSA-PSS trên dữ liệu sensor.
- Bước 5 - Thử nghiệm: chạy chương trình trên dữ liệu gốc và dữ liệu đã bị sửa 1 trường để quan sát sự thay đổi của hash/HMAC/chữ ký, ghi log kết quả (Chương 4).
- Bước 6 - Đánh giá: đối chiếu kết quả với tiêu chí đo được ở mục 3.5, phân tích rủi ro bảo mật (Chương 5) và tổng kết (Chương 6).

3.2. Mô hình / kiến trúc đề xuất

Dựa trên kiến trúc 3 vùng tin cậy ở Hình 2.1, mô hình đề xuất xác định rõ kỹ thuật mật mã áp dụng ở từng vị trí trong luồng dữ liệu, thay vì áp dụng một kỹ thuật duy nhất cho toàn hệ thống:

Hình 3.1 — Mô hình đề xuất: áp dụng kỹ thuật mật mã theo từng giai đoạn



3 mục tiêu bảo mật đạt được: Bí mật (②) — Toàn vẹn (①②③④) — Xác thực/Không thể chối bỏ (①④)

Hình 3.1 - Mô hình đề xuất: áp dụng kỹ thuật mật mã theo từng giai đoạn

Lựa chọn thiết kế: (1) HMAC được đặt ngay tại payload ở tầng ứng dụng (không chỉ dựa vào TLS ở tầng vận chuyển) vì TLS có thể bị chấm dứt sớm (TLS termination) tại gateway - nếu chỉ dựa vào TLS, dữ liệu vẫn có thể bị gateway sửa đổi mà không bị phát hiện ở phía cloud; (2) AES-GCM được dùng khi cần giấu cả nội dung, không chỉ xác thực; (3) TLS bảo vệ toàn bộ kênh gateway↔cloud nơi tài nguyên đủ mạnh để chịu chi phí bắt tay (handshake); (4) chữ ký số chỉ áp dụng cho sự kiện quan trọng, không phổ biến (firmware/OTA), vì chi phí tính toán cao hơn nhiều so với HMAC.

3.3. Môi trường, công cụ, dữ liệu

Bảng 3.3. Môi trường, công cụ, dữ liệu

Phần mềm / phần cứng	Phiên bản	Vai trò
Python	3.x	Ngôn ngữ cài đặt chương trình minh họa mật mã

Phần mềm / phần cứng	Phiên bản	Vai trò
Thư viện cryptography (PyPI)	Bản mới nhất	Cung cấp AES-256-GCM và RSA-PSS
hashlib / hmac (thư viện chuẩn Python)	Built-in	Cài đặt SHA-256 và HMAC-SHA256
sensor_data.json (dữ liệu tự tạo)	-	Dữ liệu cảm biến giả lập làm đầu vào thống nhất
Mbed TLS / TinyCrypt (tham chiếu)	Bản mới nhất trên GitHub	Đối chiếu cách triển khai thực tế trên thiết bị nhúng (không build trực tiếp trong phạm vi đồ án)

3.4. Quy trình triển khai / đánh giá

Bảng 3.4. Quy trình triển khai/đánh giá

Bước	Mốc thời gian	Sản phẩm / minh chứng
1. Khảo sát & thiết kế	Tuần 1-3	Đề cương, Chương 1-3
2. Cài đặt chương trình mật mã	Buổi 04	crypto_demo.py, sensor_data.json
3. Chạy thử nghiệm & thu log	Buổi 04	crypto_demo_log.txt
4. Đánh giá rủi ro bảo mật	Buổi 05	Chương 5
5. Hoàn thiện & rà soát toàn văn	Buổi 06	Bản nộp cuối kỳ (.docx + .pdf)

3.5. Tiêu chí đánh giá

Tỷ lệ kịch bản cho kết quả ACCEPT/REJECT đúng như thiết kế: mục tiêu 8/8 kịch bản (100%) trong Bảng 4.1 đạt kết quả như kỳ vọng.

Chương trình chứng minh được tính avalanche effect của hash (một ký tự thay đổi làm hash/HMAC thay đổi hoàn toàn) qua log thực tế, không chỉ mô tả lý thuyết.

Số lỗi triển khai mật mã phổ biến được nhận diện và có biện pháp khắc phục cụ thể: tối thiểu 8 lỗi (Bảng 5.4).

Đối chiếu được với tối thiểu 2 nhóm yêu cầu của OWASP ISVS (V4 - Communication, V8 - Cryptography) trong phần đánh giá bảo mật.

CHƯƠNG 4. TRIỂN KHAI VÀ KẾT QUẢ

4.1. Chuẩn bị môi trường

Môi trường thử nghiệm: Python 3, thư viện chuẩn hashlib/hmac và thư viện cryptography (cài đặt qua pip). Không cần thiết bị phần cứng thật vì đề tài mô phỏng ở mức thuật toán/dữ liệu; các thư viện tham chiếu Mbed TLS/TinyCrypt chỉ dùng để đối chiếu cách triển khai chuẩn công nghiệp, không build trực tiếp trong phạm vi lab của đề án này.

4.2. Thành phần sản phẩm

Thành phần sản phẩm cốt lõi của đề tài bao gồm script mã nguồn Python cài đặt 4 thuật toán mật mã (crypto_demo.py), bộ dữ liệu mô phỏng (sensor_data.json) và bảng đối chiếu các kịch bản thử nghiệm bảo mật. Toàn bộ mã nguồn, tài liệu, slide bảo vệ và minh chứng được tổ chức lưu trữ trên kho chứa (repository) GitHub theo cấu trúc sau:

repo-detai36/

```
|— configs/      # Cấu hình môi trường chạy script (nếu có)
|— data/         # Chứa file sensor_data.json (dữ liệu cảm biến giả lập)
|— report/       # Chứa file báo cáo tiểu luận cuối kỳ (.docx và .pdf)
|— results/      # Chứa log kết quả chạy (crypto_demo_log.txt) và các hình ảnh
minh chứng
|— src/          # Chứa mã nguồn cài đặt các kỹ thuật mật mã (crypto_demo.py)
|— README.md     # File mô tả thông tin đề tài, hướng dẫn cài đặt và cấu trúc
thư mục
```

Vai trò của từng thư mục:

- **configs/**: Lưu trữ các file cấu hình môi trường hoặc các thông số mặc định để chạy chương trình.

- **data/**: Chứa dữ liệu đầu vào chuẩn hóa (dữ liệu sensor giả lập dạng JSON) dùng chung cho mọi kịch bản thử nghiệm, giúp đảm bảo tính nhất quán.
- **report/**: Nơi lưu trữ bản nộp báo cáo cuối cùng đã hoàn thiện định dạng Word và PDF.
- **results/**: Nơi lưu lại các bằng chứng thực nghiệm bao gồm file log xuất ra từ màn hình console và các hình ảnh chụp kiến trúc/kết quả để trích dẫn trực tiếp vào báo cáo.
- **slides/**: Lưu trữ bản trình chiếu (slide) dùng cho Buổi 06 bảo vệ cuối kỳ trước hội đồng.
- **src/**: Thư mục mã nguồn quan trọng nhất, chứa script logic thực thi các kỹ thuật Hash, HMAC, AES-GCM và Chữ ký số.
- **README.md**: Đóng vai trò là trang chủ của repo, hướng dẫn người xem/giảng viên hiểu tổng quan về đề tài và cách chạy mã nguồn.

4.3. Kịch bản thử nghiệm

Bảng 4.3. Kịch bản thử nghiệm

ID	Mô tả	Đầu vào	Kết quả kỳ vọng
KB-01	Xác minh hash trên dữ liệu gốc	Dữ liệu sensor gốc + hash gốc	ACCEPT (khớp)
KB-02	Phát hiện thay đổi qua hash sau khi sửa 1 trường	Dữ liệu bị sửa temperature_c: 27.4→99.4	Hash thay đổi hoàn toàn (avalanche effect)
KB-03	Xác minh HMAC với dữ liệu gốc + khóa đúng	Dữ liệu gốc, HMAC gốc, khóa đúng	ACCEPT (True)

ID	Mô tả	Đầu vào	Kết quả kỳ vọng
KB-04	Xác minh HMAC trên dữ liệu bị sửa với MAC cũ	Dữ liệu sửa + HMAC gốc	REJECT (False)
KB-05	Kẻ tấn công giả mạo HMAC bằng khóa đoán mò	Dữ liệu sửa + khóa ngẫu nhiên của attacker	REJECT - server dùng khóa thật xác minh = False
KB-06	Xác minh chữ ký số RSA-PSS trên dữ liệu gốc	Dữ liệu gốc + signature + public key	ACCEPT (True)
KB-07	Xác minh chữ ký số trên dữ liệu bị sửa (tấn công giả mạo firmware/log)	Dữ liệu sửa + signature cũ + public key	REJECT (False)
KB-08	Mã hóa/giải mã AES-GCM round-trip	Dữ liệu gốc, khóa AES, nonce ngẫu nhiên	Giải mã khớp dữ liệu gốc (True)

4.4. Kết quả

Bảng 4.2 đối chiếu kết quả thực tế (trích từ crypto_demo_log.txt) với kết quả kỳ vọng ở Bảng 4.1 - toàn bộ 8/8 kịch bản đạt kết quả đúng như thiết kế, đáp ứng tiêu chí 3.5:

Bảng 4.4. Kết quả

ID	Kết quả thực tế (log)	Đạt/Chưa đạt
KB-01	SHA-256 hash gốc: 527fd0d5fe7c1174d19ee7b2c21ec2a5faab211adebc4aa 3866f1b18f7fd70ae	Đạt

ID	Kết quả thực tế (log)	Đạt/Chưa đạt
KB-02	Hash sau sửa: 83e43243f8dfd9abcb5d756d570930796df72416641aef0 2bca513fc63beab18 (khác hoàn toàn hash gốc)	Đạt
KB-03	Xác minh chữ ký với dữ liệu gốc: True	Đạt
KB-04	Kết quả xác minh HMAC trên dữ liệu bị sửa: False (phải là False)	Đạt
KB-05	Server dùng khóa thật để xác minh HMAC giả của attacker: False (phải là False)	Đạt
KB-06	Xác minh chữ ký với dữ liệu gốc: True	Đạt
KB-07	Xác minh chữ ký RSA trên dữ liệu đã sửa: False (phải là False)	Đạt
KB-08	Giải mã lại khớp dữ liệu gốc: True	Đạt

```
MINGW64/c/Users/ADMIN/OneDrive/Máy tính/231A010774-NguyenPhuocHau-DeTai36-BaoMatIoT/src
ADMIN@MINGW64 ~/OneDrive/Máy tính/231A010774-NguyenPhuocHau-DeTai36-BaoMatIoT/src (main)
$ cp ../data/sensor_data.json .
ADMIN@MINGW64 ~/OneDrive/Máy tính/231A010774-NguyenPhuocHau-DeTai36-BaoMatIoT/src (main)
$ ls
crypto_demo.py  draw_diagram.py  sensor_data.json
ADMIN@MINGW64 ~/OneDrive/Máy tính/231A010774-NguyenPhuocHau-DeTai36-BaoMatIoT/src (main)
$ python crypto_demo.py
=====
BƯỚC A: ĐỌC LƯU GỐC
=====
{
  "device_id": "SENSOR-MQSS-014",
  "location": "warehouse-A-zone3",
  "timestamp": "2026-07-28T09:11:32Z",
  "readings": {
    "temperature_c": 27.4,
    "humidity_percent": 81.2,
    "battery_voltage": 3.71
  },
  "sequence_number": 10345
}

SHA-256 hash : 527f6dd5f0c1174d19ee7b2c21ec2afab211ad6bc4aa3866f3b18f7f470ae
HMAC-SHA256 : efab02352bef5ee6029c76ca84b10ca22986c8a0f2c2e9c72c1f9ab
RSA signature (rút gọn): 97c23be77f4ed79ca448fc3ff8db727a6c16044d4a5c184d11ad31ef86381...
Xác minh chữ ký với dữ liệu gốc: True

AES-GCM ciphertext (rút gọn): c86ed8b76edd64c20a527e430f2064908037bdc02941b6439dbf3ee95331...
Ghi mã 1, 1: Nhập dữ liệu gốc: True

=====
BƯỚC B: TẤN CÔNG GIẢ LẬP - SỬA 1 KÝ TỰ TRONG DỮ LIỆU
(với ký tấn công chỉ sửa temperature_c từ 27.4 -> 99.4 để đánh lừa hệ thống)
=====
{
  "device_id": "SENSOR-MQSS-014",
  "location": "warehouse-A-zone3",
  "readings": {
    "battery_voltage": 3.71,
    "humidity_percent": 81.2,
    "temperature_c": 99.4
  },
  "sequence_number": 10345,
  "timestamp": "2026-07-28T09:11:32Z"
}

SHA-256 hash (sau khi sửa): 83e43243f6df9abcb5d756d570930796df72416641ae02bca13fc63beab18
HMAC-SHA256 (sau khi sửa): 4c778f5e1a016f79c45b9f8ed826b9b14055dudf8acfd906be92ff637

>>> Hash gốc : 527f6dd5f0c1174d19ee7b2c21ec2afab211ad6bc4aa3866f3b18f7f470ae
>>> Hash mới : 83e43243f6df9abcb5d756d570930796df72416641ae02bca13fc63beab18
>>> Hash có thay đổi: hoàn toàn không! CO (Goal:change effec)

>>> Server nhận dữ liệu BẮT B. Sửa nhưng HMAC cũ (mã) đi kèm ->
>>> Kết quả xác minh HMAC: False (phải là False)

>>> Xác minh chữ ký RSA (sign ký trên dữ liệu gốc) với dữ liệu BẮT B:
>>> Kết quả: False (phải là False)

=====
BƯỚC C: KÈ TẤN CÔNG KHÔNG CÓ KHÓA - KHÔNG THỂ GIẢ MẠO HMAC NGAY LẬP
=====
Attacker từ tạo HMAC với khóa ĐOẠN MỎI 17682a04467b04f6bdecf8aaddf779f81179c824c204c194cf01d0f9c6e61c48d
Server dùng khóa THẬT để xác minh: False (phải là False)
=> Chứng minh: chỉ hash không đủ (ai cũng tính được), phải cần HMAC/chữ ký (cần khóa bí mật).

ADMIN@MINGW64 ~/OneDrive/Máy tính/231A010774-NguyenPhuocHau-DeTai36-BaoMatIoT/src (main)
$
```

Hình 4.1 Kết quả chạy chương trình

4.5. Script sinh báo cáo (chương trình minh họa)

Script `src/crypto_demo.py` chạy độc lập, không cần tham số dòng lệnh: đọc dữ liệu từ `data/sensor_data.json`, in ra 3 bước (A) tính hash/HMAC/chữ ký/mã hóa trên dữ liệu gốc, (B) mô phỏng tấn công sửa 1 trường dữ liệu và xác minh lại, (C) mô phỏng kẻ tấn công không có khóa thật cố tạo HMAC giả. Cách chạy: `python3 crypto_demo.py > crypto_demo_log.txt`. Toàn bộ log được lưu và trích dẫn ở mục 4.4 nêu trên.

4.6. Thảo luận và hạn chế

4.6.1. Kết quả cho thấy:

Chỉ riêng hash không đủ để chống giả mạo (vì ai cũng tính lại được), phải kết hợp với khóa bí mật (HMAC) hoặc khóa bất đối xứng (chữ ký số) mới đạt được xác thực nguồn gốc - đúng như mục tiêu MT-01, MT-03 đề ra. Mô hình đề xuất ở Hình 3.1 (MT-02) được minh chứng phù hợp qua việc mỗi kỹ thuật đều thể hiện đúng vai trò khi thử nghiệm.

4.6.2. Hạn chế:

(1) Thử nghiệm chạy trên máy tính thông thường, chưa triển khai trên vi điều khiển thật nên chưa đo được chi phí tài nguyên (RAM/thời gian thực) như trên thiết bị nhúng; (2) chỉ mô phỏng 1 luồng dữ liệu, chưa xử lý tình huống nhiều thiết bị đồng thời hoặc key rotation thực tế; (3) chưa tích hợp TLS thực (chỉ mô tả lý thuyết ở Chương 2-3), phần này để trong hướng phát triển ở Chương 6.

CHƯƠNG 5. ĐÁNH GIÁ BẢO MẬT

5.1. Tài sản và dữ liệu

Bảng 5.1. Tài sản/giá trị/yêu cầu

Tài sản	Mô tả	Yêu cầu C / I / A
Dữ liệu cảm biến	Số liệu nhiệt độ, độ ẩm, điện áp pin gửi từ node	I cao · C trung bình · A cao
Khóa HMAC / AES (đối xứng)	Khóa bí mật dùng chung giữa node và server	C rất cao · I cao
Cặp khóa RSA/ECDSA	Private key ở nhà sản xuất, public key ở thiết bị	C rất cao (private) · A cao
Firmware thiết bị	Chương trình chạy trên sensor node	I rất cao · A cao
Kênh truyền gateway-cloud	Đường truyền dữ liệu qua mạng	C cao · I cao
Danh tính thiết bị (device_id)	Định danh duy nhất mỗi sensor node	I cao · A cao

5.2. Môi đe dọa và lỗ hổng (ánh xạ STRIDE)

Bảng 5.2. Môi đe dọa và lỗ hổng

Đe dọa (STRIDE)	Lỗ hổng liên quan	Điều kiện xảy ra
Spoofing (giả mạo)	Không xác thực nguồn gói tin	Thiếu HMAC/chữ ký, dùng khóa mặc định
Tampering (sửa đổi)	Dữ liệu bị sửa trên đường truyền	Không có hash/HMAC bảo vệ toàn vẹn

Đe dọa (STRIDE)	Lỗ hổng liên quan	Điều kiện xảy ra
Repudiation (chối bỏ)	Firmware/update không được ký	Không dùng chữ ký số, không lưu log
Information Disclosure (rò rỉ)	Dữ liệu truyền dạng plaintext	Không mã hóa (thiếu TLS/AES-GCM)
Denial of Service	Không giới hạn tốc độ xác minh	Không có rate limiting, dễ bị làm nghẽn
Elevation of Privilege	Quản lý khóa yếu	Dùng chung 1 khóa cho toàn hệ thống, hard-code key

5.3. Ma trận rủi ro

Bảng 5.3. Ma trận rủi ro

Mã rủi ro	Đe dọa	Khả năng (1-5)	Ảnh hưởng (1-5)	Mức độ	Ưu tiên
R1	Spoofing (giả mạo node)	4	4	16	Cao
R2	Tampering dữ liệu cảm biến	4	4	16	Cao
R3	Repudiation (firmware không ký)	2	5	10	Trung bình-cao
R4	Information Disclosure	3	3	9	Trung bình

Mã rủi ro	Đe dọa	Khả năng (1-5)	Ảnh hưởng (1-5)	Mức độ	Ưu tiên
R5	Denial of Service (xác minh)	3	3	9	Trung bình
R6	Elevation of Privilege (khóa dùng chung)	2	5	10	Trung bình-cao

5.4. Biện pháp giảm thiểu

R1, R2 (Spoofing, Tampering) → bắt buộc HMAC hoặc chữ ký số cho mỗi gói tin, mỗi thiết bị dùng khóa/định danh riêng (không dùng khóa dùng chung cho cả lô thiết bị).

R3 (Repudiation) → bắt buộc ký số firmware trước khi phát hành OTA, chỉ chấp nhận cập nhật có chữ ký hợp lệ (secure boot / verified boot).

R4 (Information Disclosure) → dùng AES-256-GCM cho payload nhạy cảm và TLS cho kênh gateway-cloud, không truyền dữ liệu ở dạng plaintext.

R5 (Denial of Service) → giới hạn tốc độ xác minh (rate limiting), thiết kế fail-closed (từ chối mặc định khi xác minh thất bại thay vì fail-open).

R6 (Elevation of Privilege) → không hard-code khóa trong firmware, lưu khóa trong secure element/TPM nếu có, thiết lập chu kỳ rotate khóa định kỳ.

Các biện pháp trên được đối chiếu với OWASP ISVS nhóm yêu cầu V4 (Communication - yêu cầu mã hóa kênh truyền, xác thực endpoint) và V8 (Cryptography - yêu cầu quản lý khóa an toàn, không tự chế thuật toán, không hard-code key).

5.5. Rủi ro còn lại

Sau khi áp dụng các biện pháp trên, một số rủi ro vẫn còn tồn tại ngoài phạm vi của đề tài: (1) tấn công kênh bên (side-channel attack) như đo dòng điện tiêu thụ hoặc thời gian tính toán để suy ra khóa trên phần cứng thật - chỉ có thể phát hiện khi triển khai trên thiết bị vật lý, không thể hiện qua mô phỏng phần mềm; (2) tấn công vật lý trực tiếp lên chip (chip tampering, dump firmware qua JTAG) nằm ngoài phạm

vi mô phỏng của đồ án; (3) quản lý vòng đời khóa ở quy mô lớn (hàng nghìn thiết bị) đòi hỏi hạ tầng PKI/HSM đầy đủ, vượt quá phạm vi lab hiện tại. Các rủi ro này cần được xem xét lại khi đề tài mở rộng sang triển khai thiết bị thật (xem Chương 6.3).

CHƯƠNG 6. KẾT LUẬN

6.1. Kết luận theo mục tiêu

MT-01 - Đạt: Chương 2 đã giải thích rõ vai trò riêng biệt của hash, HMAC, mã hóa đối xứng AEAD và chữ ký số đối với 3 mục tiêu bảo mật, có bảng so sánh định lượng (Bảng 2.1).

MT-02 - Đạt: Chương 3 xây dựng mô hình đề xuất (Hình 3.1) xác định rõ vị trí áp dụng từng kỹ thuật mật mã trong luồng dữ liệu thực tế của đề tài.

MT-03 - Đạt: Chương 4 cài đặt và chạy thành công chương trình Python với 8/8 kịch bản thử nghiệm cho kết quả đúng như thiết kế, có log minh chứng cụ thể.

MT-04 - Đạt: Chương 5 xây dựng bảng tài sản, ánh xạ STRIDE, ma trận rủi ro 5×5 và đề xuất biện pháp giảm thiểu tương ứng cho từng rủi ro, đối chiếu OWASP ISVS.

6.2. Hạn chế

Toàn bộ thử nghiệm chạy trên máy tính thông thường, chưa triển khai và đo hiệu năng thực tế trên vi điều khiển/thiết bị nhúng thật.

Phạm vi mô phỏng hẹp: chỉ 1 luồng dữ liệu cảm biến, chưa xử lý tình huống nhiều thiết bị đồng thời hoặc key rotation trong vận hành thực tế.

Ma trận rủi ro ở Chương 5 mang tính định tính/minh họa dựa trên đánh giá chủ quan của người thực hiện, chưa dựa trên dữ liệu tấn công thực tế.

6.3. Hướng phát triển

Triển khai thực tế trên thiết bị nhúng (ví dụ ESP32) sử dụng Mbed TLS hoặc TinyCrypt, đo hiệu năng và tài nguyên tiêu thụ thực tế.

Tích hợp TLS thực với chứng chỉ X.509 và cơ chế certificate pinning cho thiết bị IoT.

Xây dựng dashboard giám sát cảnh báo khi phát hiện gói tin xác minh thất bại (tích hợp cùng cơ chế fail-closed ở mục 5.4).

Mở rộng số lượng thiết bị mô phỏng và dữ liệu, bổ sung hạ tầng PKI để quản lý vòng đời khóa tự động ở quy mô lớn hơn.

DANH MỤC TÀI LIỆU THAM KHẢO

- [1] Mbed-TLS. Mbed TLS - SSL/TLS and cryptography library. GitHub. <https://github.com/Mbed-TLS/mbedtls> - truy cập 28/07/2026 - dùng cho mục 2.3, 2.4, 3.3.
- [2] Intel. TinyCrypt - small footprint cryptographic library. GitHub. <https://github.com/intel/tinycrypt> - truy cập 28/07/2026 - dùng cho mục 2.3, 2.4, 3.3.
- [3] OWASP. IoT Security Verification Standard (ISVS). GitHub. <https://github.com/OWASP/IoT-Security-Verification-Standard-ISVS> - truy cập 28/07/2026 - dùng cho Chương 5.
- [4] National Institute of Standards and Technology (NIST). FIPS 198-1, The Keyed-Hash Message Authentication Code (HMAC), 2008. <https://csrc.nist.gov/pubs/fips/198-1/final> - truy cập 28/07/2026 - dùng cho mục 2.2, 2.3.
- [5] Dworkin, M. NIST SP 800-38D, Recommendation for Block Cipher Modes of Operation: Galois/Counter Mode (GCM) and GMAC, 2007. <https://www.nist.gov/publications/recommendation-block-cipher-modes-operation-galoiscounter-mode-gcm-and-gmac> - truy cập 28/07/2026 - dùng cho mục 2.3, Chương 4.
- [6] IETF. RFC 8446 - The Transport Layer Security (TLS) Protocol Version 1.3, 2018. <https://www.rfc-editor.org/rfc/rfc8446> - truy cập 28/07/2026 - dùng cho mục 2.3, 3.2.

PHỤ LỤC - PHẦN A: CHECKLIST TIẾN ĐỘ THEO TUẦN

Bảng checklist tiến độ theo tuần

Tuần	Trạng thái	Minh chứng
Tuần 01 - Đề cương	Đạt	Đã nộp: NguyenPhuocHau_baocaotieuluanlan01.docx
Tuần 02 - Chương 1 + nháp Chương 2	Đạt	Tr. viii–x (Chương 1, file này) + nộp cùng Lần 1
Tuần 03 - Chương 2-3 hoàn chỉnh	Đạt	Tr. xi–xix (Chương 2 mục 2.1–2.6, Chương 3 mục 3.1–3.5)
Buổi 04 - Chương 4	Đạt	Tr. xx–xxiii (Chương 4) + repo: src/crypto_demo.py, results/crypto_demo_log.txt
Buổi 05 - Chương 5-6	Đạt	Tr. xxiv–xxvii (Chương 5–6)
Buổi 06 - Rà soát & nộp bản hoàn chỉnh	Đạt	Đã rà soát tất cả và hoàn thiện tiểu luận